

04_ON_DEVICE_INGESTION_AND_W5H_FRAMEWORK.md

On-Device Markdown Ingestion, JSONL Marker Extraction & The W5+H Operational Framework

1. Document Scope & Purpose

This specification establishes the on-device ingestion pipeline and codifies the **W5+H System Orientation** across all agents, tools, and repositories. It unifies and standardizes:

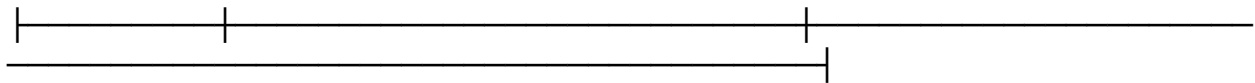
- GLM-PT.1-MASTER.DOCUMENT.md & GLM-PT.2-fusion-response.md
- JSONL labeling & Personal Wiki automatic file compiler
- W5/H-(6-files) (User explains audit.txt, What and Why.txt, What and How.txt, Where and When.txt)

2. The W5+H Operational Matrix

The W5+H framework organizes every technical action, tool execution, and architecture document in the system:

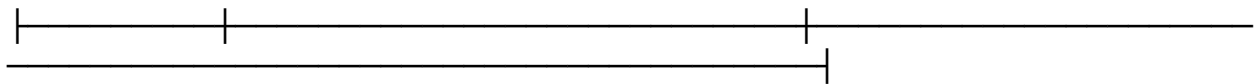
THE W5+H SYSTEM MATRIX			
Vector	Core Architectural Query	System Implementation Definition	
WHO daemon),	Which agent or daemon?	Horizons UI (conciERGE), ÆEsc (terminal/ADB	

		Æyre (media daemon), Qwen 3.5 0.8B (executor),
		Qwen 3.5 9B (query), Official Enterprise Auditor,
		or Incognito Red Auditor.



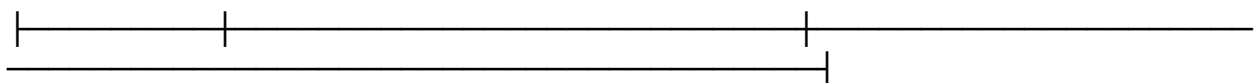
| WHAT | What component or tool? | Pure Markdown notes, atomic JSONL markers,
GGUF weights, |

| | | SQLite tables, PostgreSQL OB1 records, ECC skills, |
| | | or Prime Agent RLM functions. |



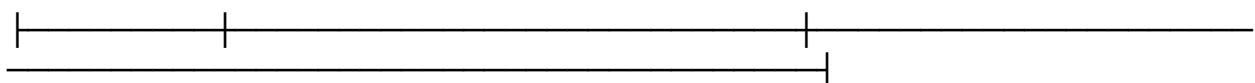
| WHEN | What triggers the action? | User mic tap, screen change via Media SDK, periodic
cron |

| | | sweep, end-of-day P2P sync, or RLVR verifier failure (-1.0) |



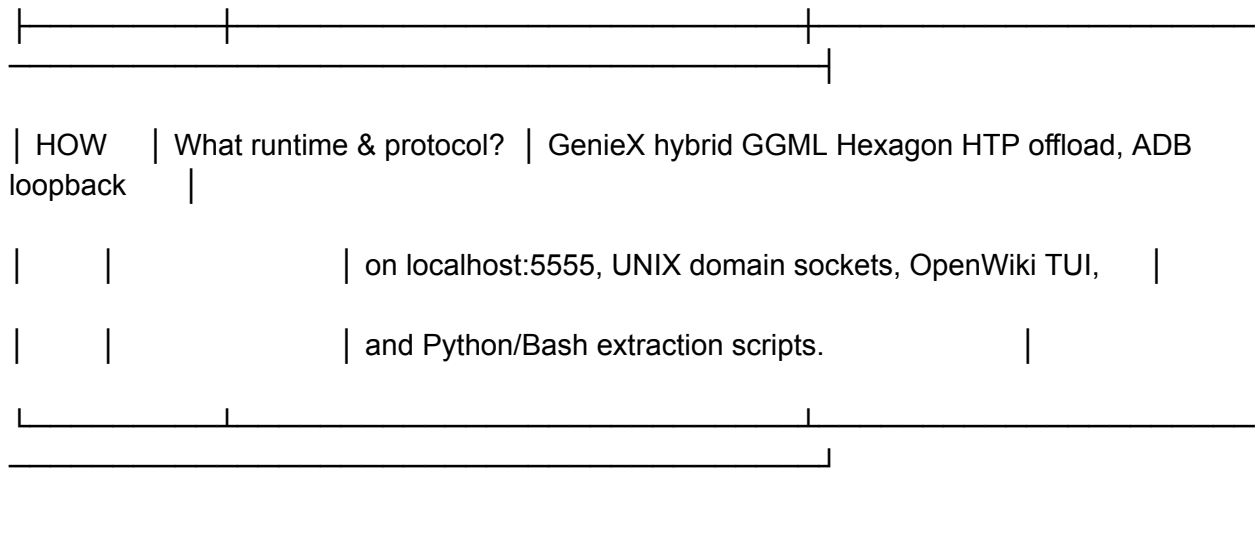
| WHERE | Where does it run & live? | Node Alpha (Moto RAZR Ultra 2025), Node Beta
(Jetson), |

| | | Node Gamma (Rubik Pi 3), or Directory Tier (01 through 05) |



| WHY | Why this specific design? | Prevent Android LMK eviction, maintain zero-trust
bounds, |

| | | zero-copy memory throughput, prevent context bloat, |
| | | and enforce sensory immutability. |



3. The On-Device Ingestion Pipeline: Markdown-First & JSONL Markers

The Problem It Solves

Raw files dumped into a workspace come in mixed formats (DOCX, HTML, TXT, PDF). If an AI model attempts to read raw binaries or HTML web code, tokens are wasted on formatting tags and timestamps, and context windows choke. Furthermore, generating PDFs on a phone burns CPU and wastes storage. **The system standard is strictly clean Markdown (.md) and machine-readable JSONL (.jsonl).**

[Raw Files Dropped in ~/raw-bucket/]

|



[convert_raw_to_markdown.py] (Termux / ÆEsc)

- Strips web UI fluff, HTML tags, and styling code
- Extracts text from DOCX, TXT, and Markdown
- Prepends strict YAML frontmatter metadata

|



[Output: 01_raw_sources/]

|



[generate_jsonl_markers.py]

- Generates atomic SHA256 content hashes
- Extracts top 15 retrieval keyword tokens
- Writes record to manifest.jsonl

|



[OpenWiki CLI Synthesis]

- GLM 5.2 via OpenRouter reads new clean markdown
- Generates bidirectional [[wiki_links]] in 02_wiki_md/

4. Production Script: `convert_raw_to_markdown.py`

Runs directly in Termux / AEsc using Python 3 with beginner-proof comments.

```
#!/usr/bin/env python3
```

```
"""
```

```
convert_raw_to_markdown.py
```

```
-----
```

PURPOSE FOR BEGINNER DEVELOPERS:

This script monitors an incoming folder of messy notes, downloads, or web scrapes and converts them into clean, standardized Markdown files.

It strips away HTML tags and web interface junk so local AI models only read pure text.

```
"""
```

```
import sys
```

```
from pathlib import Path
```

```
from datetime import datetime, timezone
```

```
import markdownify
```

```
from docx import Document
```

```
RAW_DIR = Path("~/raw-bucket").expanduser()
```

```
OUTPUT_DIR = Path("~/novae-xorpus/01_raw_sources/text").expanduser()
```

```
def convert_docx(path: Path) -> str:
```

```
    """Reads Microsoft Word .docx files and converts headings and paragraphs to Markdown."""
```

```
    doc = Document(str(path))
```

```
    lines = []
```

```
    for p in doc.paragraphs:
```

```
        if p.style.name.startswith("Heading"):
```

```
            try:
```

```
                lvl = int(p.style.name.replace("Heading ", ""))
```

```
                lines.append(f"{'#' * lvl} {p.text}")
```

```
            except ValueError:
```

```
                lines.append(p.text)
```

```
        elif p.text.strip():
```

```

        lines.append(p.text)

    return "\n\n".join(lines)

def convert_html(path: Path) -> str:

    """Strips HTML tags and converts web content into clean Markdown."""

    raw = path.read_text(encoding="utf-8", errors="ignore")

    return markdownify.markdownify(raw, heading_style="ATX")

def convert_txt(path: Path) -> str:

    """Wraps plain text files in a top-level Markdown header."""

    raw = path.read_text(encoding="utf-8", errors="ignore")

    return f"# {path.stem}\n\n{raw}"

CONVERTERS = {

    ".docx": convert_docx,

    ".html": convert_html,

    ".htm": convert_html,

    ".txt": convert_txt,

    ".md": lambda p: p.read_text(encoding="utf-8", errors="ignore")

}

def main():

    if not RAW_DIR.exists():

        print(f"[!] Raw directory {RAW_DIR} does not exist.")

        return

    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

```

```

count = 0

for file in RAW_DIR.iterdir():

    ext = file.suffix.lower()

    if ext in CONVERTERS:

        out_file = OUTPUT_DIR / f'{file.stem}.md'

        print(f"[*] Converting: {file.name} -> {out_file.name}")

        try:

            body = CONVERTERS[ext](file)

            frontmatter = f'""""---

source_file: {file.name}

ingested_at: {datetime.now(timezone.utc).isoformat()}

tier: 1

---

""""

            out_file.write_text(frontmatter + body, encoding="utf-8")

            count += 1

        except Exception as e:

            print(f"[!] Error converting {file.name}: {e}")

        print(f"[✓] Converted {count} files to clean Markdown in {OUTPUT_DIR}")

if __name__ == "__main__":

    main()

```

5. Production Script: `generate_jsonl_markers.py`

Extracts atomic JSONL index records without requiring heavy cloud databases.

```
#!/usr/bin/env python3
```

```
"""
```

```
generate_jsonl_markers.py
```

```
-----
```

PURPOSE FOR BEGINNER DEVELOPERS:

For an AI agent to search through hundreds of files without crashing phone memory, it cannot open every file. It needs an index file (manifest.jsonl).

This script reads every Markdown file in your vault, grabs its title, first paragraph, calculates a unique SHA256 fingerprint, extracts 15 keyword tokens, and writes a single JSON line for each file into manifest.jsonl.

```
"""
```

```
import json
```

```
import hashlib
```

```
from pathlib import Path
```

```
from datetime import datetime
```

```
VAULT_DIR = Path("~/novae-xorpus").expanduser()
```

```
OUTPUT_FILE = VAULT_DIR / "manifest.jsonl"
```

```
def make_marker(file_path: Path) -> dict:
```

```
    content = file_path.read_text(encoding="utf-8", errors="ignore")
```

```

title = file_path.stem

summary = ""

tokens = set()

for line in content.split("\n"):

    stripped = line.strip()

    if not stripped or stripped.startswith("---"):

        continue

    if stripped.startswith("# ") and title == file_path.stem:

        title = stripped.lstrip("# ").strip()

    elif not summary and not stripped.startswith("#"):

        summary = stripped[:200]

    if stripped.startswith("##"):

        for w in stripped.lstrip("# ").lower().split():

            if len(w) > 3 and w.isalnum():

                tokens.add(w)

h = hashlib.sha256(content.encode("utf-8")).hexdigest()[:16]

return {

    "record_id": f"REC_{h}_{file_path.stem.upper()[:20]}",

    "path": str(file_path.relative_to(VAULT_DIR)),

    "sha256": h,

    "title": title,

```

```

        "summary": summary or "Technical reference document.",

        "tokens": list(tokens)[:15],

        "last_modified": datetime.fromtimestamp(file_path.stat().st_mtime).isoformat()

    }

def main():

    markers = []

    for md in VAULT_DIR.rglob("*.md"):

        if ".git" in str(md):

            continue

        try:

            markers.append(make_marker(md))

        except Exception as e:

            print(f"[!] Error on {md.name}: {e}")

    with open(OUTPUT_FILE, "w", encoding="utf-8") as f:

        for m in markers:

            f.write(json.dumps(m, ensure_ascii=False) + "\n")

    print(f"[✓] Generated {len(markers)} JSONL markers in {OUTPUT_FILE}")

if __name__ == "__main__":

    main()

```